

Microfrontales I

Qué son los MFEs, cómo delimitarlos con Domain-Driven Design y qué herramientas los construyen

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Miguel Ángel Muñoz

Contenido

1. Qué son los microfrontends.....	3
2. Cómo se despliegan los MFEs: compilación y ejecución.....	3
3. Por qué migrar a microfrontales: del monolito al polyolith.....	4
4. Domain-Driven Design	4
4.1. Diseño táctico y diseño estratégico.....	5
4.2. En busca de los dominios	5
4.3. Modelando los dominios.....	5
5. Agrupadores de módulos	6
5.1. Qué hacen y por qué son necesarios	6
5.2. Webpack: entry, output, loaders y plugins	6
6. Qué hemos aprendido.....	7

1. Qué son los microfrontales

Los microfrontends (MFEs) surgen como el equivalente, en la capa de presentación, a los microservicios: si los microservicios permiten dividir la lógica de negocio de un backend en piezas pequeñas, autónomas y desplegadas por separado, los microfrontends aplican exactamente la misma filosofía al frontend. No es casualidad que también se les conozca como "los microservicios del frontal".

La idea de fondo es sencilla pero poderosa: en lugar de construir una única aplicación gigante que contenga todas las pantallas, toda la lógica de negocio y todas las llamadas a servicios, el frontend se divide en piezas independientes —los MFEs— que un equipo puede desarrollar, probar y desplegar sin depender del resto.

Esta independencia se extiende también a los datos: igual que cada microservicio debería tener su propia fuente de datos y ser stateless, cada MFE debería gestionar su propio estado y sus propias llamadas a API sin acoplarse a las de otro MFE.

De esta división nace además un beneficio organizativo: los equipos pueden especializarse en un dominio concreto sin necesidad de entender el resto de la aplicación, ya que cada equipo despliega su funcionalidad de forma completamente independiente. Eso sí, para que el conjunto funcione como un producto coherente, esos equipos deben seguir coordinándose sobre cómo encaja cada pieza en la arquitectura general.

Un ejemplo ayuda a visualizarlo: piensa en la cabecera y el pie de página de un sitio web. Ambos son transversales a todas las páginas y funcionalmente independientes del contenido que envuelven, así que son candidatos perfectos para convertirse en MFEs reutilizables.

Otro ejemplo, más cercano al negocio: la aplicación de un banco. Tras el login se muestra la "posición global", con secciones como cuentas, préstamos e inversiones. Cada una de esas secciones puede ser un MFE independiente que sabe qué APIs llamar y cómo gestionar su propio estado; en la práctica, son miniaplicaciones ensambladas en una página común.

★ Lo más importante

- ▶ Los MFEs son el equivalente de los microservicios en la capa de frontend: piezas pequeñas, autocontenidas e independientes.
- ▶ Son autocontenidos: incluyen toda su lógica de negocio, a diferencia de los web components, que deben mantenerse agnósticos al negocio.
- ▶ Se pueden desplegar y ejecutar de manera completamente independiente, incluso sueltos en el navegador.

2. Cómo se despliegan los MFEs: compilación y ejecución

Que los MFEs se puedan desplegar de manera independiente no significa que solo exista una forma de hacerlo. En la práctica hay dos estrategias con implicaciones muy distintas: desplegarlos como dependencias en tiempo de compilación o hacerlo en tiempo de ejecución.

2.1. Tiempo de compilación

El MFE se mantiene en un módulo separado, pero se incorpora a la aplicación host como una dependencia más —normalmente a través de un repositorio privado de artefactos, por ejemplo NPM—. Cada vez que el MFE cambia, la aplicación host tiene que volver a compilarse para que el cambio se refleje.

- ▶ Ventaja: la aplicación se despliega como un todo; todo el código queda contenido en una única build.
- ▶ Desventaja: los MFEs no se pueden modificar en caliente. Cada cambio exige volver a desplegar la aplicación host, y es fácil terminar con distintas versiones de un mismo MFE conviviendo en distintas aplicaciones.

2.2. Tiempo de ejecución

Cada MFE se despliega de forma independiente a producción, con su propia URL, y las aplicaciones que lo necesitan lo cargan de manera dinámica sin tenerlo compilado dentro de su propio bundle. Cualquier cambio en el MFE se refleja automáticamente en todas las aplicaciones que lo consumen, sin que estas tengan que volver a desplegarse. Para lograr esta carga dinámica se recurre a tecnologías como ECMAScript Modules (ESM) o Module Federation —esta última, una funcionalidad de webpack que veremos más adelante—.

- ▶ Ventaja: actualización inmediata en todas las aplicaciones que consumen el MFE, sin necesidad de recompilarlas.
- ▶ Desventaja: la aplicación host pierde control directo sobre el MFE; su funcionamiento completo depende de que la carga en tiempo de ejecución se complete correctamente.

Ninguna de las dos opciones es universalmente mejor: ambas tienen ventajas y desventajas claras, y lo habitual es elegir una u otra —o combinarlas— en función del caso concreto.

★ Lo más importante

- ▶ Tiempo de compilación: el MFE se integra como dependencia (vía NPM) y viaja dentro del build de la app host; sencillo de desplegar, pero sin actualización en caliente.
- ▶ Tiempo de ejecución: cada MFE tiene su propia URL y se carga de forma dinámica con ESM o Module Federation; se actualiza al instante, pero la app host depende de esa carga para funcionar

3. Por qué migrar a microfrontales: del monolito al polyolith

La mayoría de las aplicaciones nacen como un monolito: un único repositorio con un único esqueleto de aplicación que concentra páginas, componentes, lógica de negocio y llamadas a servicios. Mientras el proyecto es pequeño esto funciona bien, pero según crece el equipo el monolito empieza a pasar factura: todos los desarrolladores trabajan sobre el mismo bloque de código, los conflictos de repositorio se multiplican y un cambio mal comunicado en un punto de la aplicación puede convertirse en un infierno de depuración para otro compañero. A eso hay que sumar los tiempos, cada vez más largos, de tests, formateo y despliegue.

Cuando una empresa es consciente de este problema, el paso intermedio habitual es evolucionar hacia una arquitectura de múltiples monolitos, conocida como polyolith: se van extrayendo, una a una, las funcionalidades del monolito original, normalmente organizadas por ruta (/home, /accounts, /clients, /contact...). El resultado es un conjunto de aplicaciones que se despliegan y mantienen por separado, y es precisamente en esa separación donde empiezan a intuirse las ventajas de los microfrontales: más velocidad de desarrollo y menos conflictos entre equipos.

Para que esta subdivisión tenga sentido de verdad —y no sea solo trocear por trocear— conviene apoyarse en una estrategia de diseño que identifique los límites naturales del negocio. Esa es precisamente la función de Domain-Driven Design (DDD), que veremos en el siguiente apartado.

Conviene tener presente también cómo encajan los MFEs entre sí una vez desplegados: normalmente conviven dentro de una página que actúa de "Application Shell" (o simplemente Shell), la pieza encargada de los servicios transversales —autenticación, estado global, identidad del usuario— que cada MFE necesita para funcionar aunque sea autocontenido. Por ejemplo, un MFE de cuentas corrientes puede desplegarse de forma independiente, pero necesita que el Shell le indique de qué usuario debe consultar los datos.

★ Lo más importante

- ▶ El monolito es cómodo al principio, pero degrada la velocidad de desarrollo según crece el equipo.
- ▶ El paso intermedio típico es el polyolith: extraer funcionalidades del monolito, normalmente por ruta.
- ▶ El Application Shell gestiona lo transversal (autenticación, estado, identidad) para que los MFEs puedan seguir siendo autocontenidos.

4. Domain-Driven Design

Dividir una aplicación en piezas pequeñas por trocear sin criterio es sencillo, pero corre el riesgo de generar dominios artificiales, mal delimitados y acoplados entre sí. Domain-Driven Design (DDD) aporta precisamente ese criterio: una metodología para convertir un sistema de software complejo en un diseño de aplicación bien estructurado, buscando siempre la escalabilidad — el modelo tiene que poder sostenerse conforme la empresa crece o evoluciona.

4.1. Diseño táctico y diseño estratégico

DDD se puede dividir en dos grandes modelos:

- ▶ Diseño táctico: propone conceptos concretos de implementación, como el layering (división en capas) o patrones de diseño clásicos (patrón factoría, patrón adaptador, etc.).
- ▶ Diseño estratégico: se enfoca en identificar y subdividir el sistema en partes más pequeñas, desacopladas y menos complejas — los dominios.

En la práctica, el diseño estratégico es el que marca los límites de los futuros MFEs, mientras que el diseño táctico entra en juego dentro de cada dominio ya delimitado.

4.2. En busca de los dominios

El objetivo del diseño estratégico es identificar dominios que cumplan un único requisito: ser independientes entre sí y no necesitar saber demasiado los unos de los otros. DDD propone tres señales para detectarlos:

- ▶ Estructura de la organización: si distintos roles o departamentos son responsables de pasos diferentes de un mismo proceso, es un indicio de que existen varios subdominios.
- ▶ Vocabulario: si un mismo término del negocio se usa con significados o importancia distintos según el contexto, es probable que en realidad existan dominios distintos.
- ▶ Eventos cruciales: los puntos donde una tarea significativa se completa —y el proceso continúa en otro momento, lugar o con otros roles— también delimitan subdominios.

Un ejemplo con un banco lo aclara: al incorporar una nueva tecnología (por ejemplo, una base de datos de geolocalización de IPs para prevención de fraude), el proceso pasa por Arquitecto (propone opciones), Head (valida y elige), Dirección (aprueba herramienta y presupuesto) y Compras (obtiene presupuesto y realiza el pedido). Aplicando las señales anteriores —distintos roles responsables de pasos distintos— se pueden extraer cuatro dominios: Selección, Aprobación, Compras y Catálogo. Cada uno se puede definir de forma independiente, aunque en algún momento tengan que interactuar entre sí: el resultado de Selección alimenta a Aprobación, lo aprobado pasa a Compras y, finalmente, se registra en el Catálogo.

4.3. Modelando los dominios

Una vez identificados los dominios toca modelarlos, es decir, definir su estructura. Aquí es clave hacerlo de forma separada e independiente para cada uno, incluso a costa de asumir cierta repetición: el atajo de crear un modelo "genérico" que sirva para varios dominios a la vez suele terminar en una entidad gigante llena de datos innecesarios para cada uno de ellos.

El ejemplo típico es el de un banco con dominio de cuentas y dominio de tarjetas: ambos comparten el concepto de identidad (el cliente), pero conviene modelar cada dominio con su propia noción de cliente — con solo los datos que necesita— en lugar de forzar una identidad única y sobrecargada para ambos.

★ Regla clave

- ▶ Nunca acoples los dominios entre sí: cada uno debe definir sus propios modelos y funcionalidades sin depender de las necesidades de otro. Si otro dominio necesita interactuar con él, la vía correcta es exponer una API que dé acceso a lo estrictamente necesario, sin comprometer su independencia.

5. Agrupadores de módulos

5.1. Qué hacen y por qué son necesarios

A medida que crece una aplicación se van acumulando archivos de todo tipo —.js, .ts, .scss, .json...— tanto propios como de las dependencias que se instalan. Antiguamente había que incluir cada script por separado en el HTML, lo que terminaba en listados interminables y en librerías que casi nunca se actualizaban por lo tedioso del proceso.

Los agrupadores de módulos (module bundlers) resuelven este problema: combinan todos esos archivos en uno o varios ficheros de salida, y de paso permiten ejecutar tareas configurables muy útiles:

- ▶ Transpilación de código: por ejemplo, convertir TypeScript a JavaScript.
- ▶ Compatibilidad de versiones con herramientas como Babel, que traduce el JavaScript más moderno a la versión que necesite el navegador de destino.
- ▶ Tree shaking: elimina del bundle el código que no se usa o que está repetido, evitando arrastrar dependencias completas cuando solo hace falta una pequeña parte.
- ▶ Compilación: por ejemplo, convertir SCSS a CSS dentro del propio proceso de bundling.

Para hacer todo esto, el bundler necesita primero un entry file: el punto de partida a partir del cual construye un mapa de dependencias con todos los archivos implicados (incluso puede generarse una visualización de ese mapa, indicando el peso de cada dependencia). Una vez ejecutadas todas las tareas, genera los outputs, que pueden ser uno o varios archivos.

Existen varias alternativas de bundler —browserify, esbuild, parcel, rollup y webpack—, cada una con sus propias características. Webpack es, con diferencia, el más extendido, y además el que permite implementar module federation, la tecnología clave para cargar MFEs de forma dinámica en tiempo de ejecución.

5.2. Webpack: entry, output, loaders y plugins

La configuración de webpack se organiza en torno a unos pocos conceptos centrales:

- ▶ **Entry:** el punto de entrada. Por defecto es `./src/index.js`, pero se puede definir como un string, un array (varios puntos de entrada) o un objeto con una key por cada entry point, algo especialmente útil cuando hay varios (por ejemplo: `app` y `adminApp` apuntando cada uno a su propio archivo).
- ▶ **Output:** dónde se dejan los archivos resultantes y cómo se llaman. Por defecto es `./dist/main.js`. Cuando hay varios entry points, `filename` admite sustituciones como `'[name].js'` para no pisar unos archivos con otros. El parámetro `publicPath`, que también vive aquí, cobra especial relevancia en escenarios de `module federation`.
- ▶ **Loaders:** webpack, por sí solo, solo entiende JavaScript y JSON; los loaders son los que le permiten procesar cualquier otro tipo de archivo y convertirlo en un módulo. Cada regla se define con dos parámetros: `test` (una expresión regular que identifica la extensión del archivo) y `use` (el loader o loaders que procesarán ese tipo de archivo). Un ejemplo habitual es aplicar `style-loader`, `css-loader` y `postcss-loader` a los archivos `.css/.scss/.sass`, y `babel-loader` a los `.ts/.tsx/.js/.jsx`.
- ▶ **Plugins:** añaden funcionalidades adicionales a las que webpack trae de serie. A diferencia de los loaders, hay que instalarlos por npm y además requerirlos explícitamente en el archivo de configuración (con `require`, ya que webpack se ejecuta sobre Node.js y usa la sintaxis CommonJS). Uno de los más habituales es `html-webpack-plugin`, que recoge los HTML, los procesa e inyecta en ellos los bundles generados en el output. El `ModuleFederationPlugin` es el que hace posible exponer y consumir MFEs entre aplicaciones distintas en tiempo de ejecución.
- ▶ **DevServer:** mediante la dependencia `webpack-dev-server`, configurable directamente desde el propio `webpack.config.js` (`puerto`, `historyApiFallback...`), se puede levantar un servidor de desarrollo con un par de líneas y arrancarlo con un simple script de npm.

★ Lo más importante

- ▶ Webpack construye un mapa de dependencias a partir de un entry file y genera uno o varios archivos de salida (output).
- ▶ Los loaders enseñan a webpack a procesar archivos que no son JS/JSON; los plugins añaden funcionalidades extra y deben requerirse explícitamente.
- ▶ Webpack es la puerta de entrada a `module federation`, la pieza que permite cargar microfrontales de forma dinámica en tiempo de ejecución.

6. Qué hemos aprendido

A lo largo de este documento hemos recorrido el porqué y el cómo de dividir un frontend en microfrontales: qué son y en qué se parecen a los microservicios, por qué las aplicaciones monolíticas terminan generando fricción a medida que crecen, cómo Domain-Driven Design ayuda a encontrar los límites correctos entre dominios sin acoplarlos entre sí, y qué papel juegan los agrupadores de módulos —con webpack a la cabeza— para convertir todo ese código en artefactos desplegables.

La idea central que conviene retener es que los MFEs son una forma de subdividir el frontend en piezas autocontenidas, desplegables de manera independiente y organizadas alrededor de dominios de negocio bien definidos, no de rutas arbitrarias.

También hemos visto que no hay una única forma correcta de desplegar un MFE: elegir entre tiempo de compilación y tiempo de ejecución es una decisión de arquitectura con consecuencias reales sobre la velocidad de actualización y el control que la aplicación host mantiene sobre sus piezas.

Ahora toca practicar: coger una funcionalidad de un proyecto real (o de uno de prueba), identificar su dominio con las técnicas de DDD, extraerla como MFE y probar a montar su configuración de webpack desde cero, con su propio entry, output, loaders y plugins.